

EECS 841 Homework 3

Johnpaul Mbagwu

1

Q1

Tabela 1. Top-Left 10×10 Snippet of the Similarity Matrix

0.8302	0.5005	0.5438	0.5446	0.5528	0.5954	0.4034	0.5180	0.4021	0.4881
0.6015	0.9563	0.4528	0.5225	0.3632	0.6346	0.5756	0.5036	0.5203	0.1844
0.6255	0.4652	0.8898	0.5942	0.4264	0.4749	0.3472	0.4063	0.4855	0.3183
0.6217	0.4096	0.5793	0.8476	0.5607	0.5920	0.2934	0.6526	0.5921	0.5526
0.5934	0.2927	0.5339	0.7010	0.9626	0.6010	0.4851	0.6579	0.5509	0.8150
0.6407	0.5286	0.5322	0.6995	0.6665	0.7864	0.4783	0.6187	0.5809	0.6010
0.7587	0.6191	0.5355	0.7302	0.6417	0.7098	0.6255	0.6609	0.5245	0.5197
0.6428	0.3925	0.4208	0.6980	0.6451	0.5590	0.4266	0.8775	0.4174	0.6111
0.4076	0.4645	0.4830	0.6402	0.4283	0.5147	0.1807	0.4910	0.7298	0.4444
0.4652	0.1305	0.3572	0.6019	0.7942	0.5067	0.3580	0.6048	0.4538	0.8596

Q2: Threshold Feasibility

The lowest genuine similarity score was 0.4604, while the highest impostor similarity score was 0.8579. Ideally, in a perfectly discriminative system, all genuine scores would exceed all impostor scores, allowing for a clean threshold that separates the two classes without error.

However, the observed overlap between the distributions of genuine and impostor scores indicates that such perfect separation is not feasible in this case. Any threshold selected within the range $[0.4604, 0.8579]$ will inherently result in a trade-off: it will either reject some genuine users (false negatives) or incorrectly accept some impostors (false positives), depending on its placement.

This limitation highlights the need for a more discriminative set of features or an improved classification model to enhance class separability and reduce classification error.

Q3: Decidability Index d'

The computed decidability index, which indicates a moderate degree of class separability, was 2.70. A higher d' value, which often corresponds to better recognition performance, denotes a larger difference between the mean of the genuine and imposter score distributions in relation to their spread. A system's ability to reliably differentiate between real and fraudulent users increases with d' , ideally lowering the rates of mistaken acceptance and rejection. Practically speaking, systems are thought to function well; hence, even if 2.70 is commendable, it may still be improved.

Q4: Does RGB channels help?

RGB channels can offer more discriminative data (such as lighting and skin tone), which could enhance correlation-based matching. But because of changes in illumination, color may sometimes add noise. Therefore, multi-channel fusion may be advantageous in more varied contexts, although grayscale images are frequently more resilient in controlled conditions.

Listing 1. CODE

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Thu Apr 24 12:55:13 2025
@author: johnpaulmbagwu
"""

import os
import numpy as np
import cv2
import matplotlib.pyplot as plt

# Step 1: Define paths to gallery and probe sets
gallery_path = "/Users/johnpaulmbagwu/Downloads/GallerySet"
probe_path = "/Users/johnpaulmbagwu/Downloads/ProbeSet"

# Step 2: Validate the folder paths
if not os.path.exists(gallery_path) or not os.path.exists(probe_path):
    raise FileNotFoundError(f"Expected_folders_not_found:\n-_{gallery_path}\n-_{probe_path}")

# Step 3: Helper functions
def load_grayscale_image(path):
    return cv2.imread(path, cv2.IMREAD_GRAYSCALE).astype(np.float32)

def normalized_correlation(img1, img2):
    img1 = img1 - np.mean(img1)
    img2 = img2 - np.mean(img2)
    numerator = np.sum(img1 * img2)
    denominator = np.sqrt(np.sum(img1**2) * np.sum(img2**2))
    return numerator / denominator if denominator != 0 else 0

def build_similarity_matrix(gallery_path, probe_path):
    gallery_files = sorted(os.listdir(gallery_path))
    probe_files = sorted(os.listdir(probe_path))
    A = np.zeros((100, 100))
    for i in range(100):
        probe_img = load_grayscale_image(os.path.join(probe_path, probe_files[i]))
        for j in range(100):
            gallery_img = load_grayscale_image(os.path.join(gallery_path, gallery_files[j]))
            A[i, j] = normalized_correlation(probe_img, gallery_img)
    return A
```

```

# Step 4: Build and save similarity matrix
A = build_similarity_matrix(gallery_path, probe_path)
np.save("similarity_matrix.npy", A)

# Step 5: Analyze results
genuine = np.diag(A)
impostor = A[~np.eye(100, dtype=bool)]

lowest_genuine = np.min(genuine)
highest_impostor = np.max(impostor)

mu_g, mu_i = np.mean(genuine), np.mean(impostor)
std_g, std_i = np.std(genuine), np.std(impostor)
d_prime = abs(mu_g - mu_i) / np.sqrt(0.5 * (std_g**2 + std_i**2))

print("Lowest_Genuine_Score:", lowest_genuine)
print("Highest_Impostor_Score:", highest_impostor)
print("Decidability_Index_(d'):", d_prime)

# Step 6: Plot score distributions
plt.figure(figsize=(8, 5))
plt.hist(impostor, bins=50, alpha=0.6, label='Impostor_Scores', color='red', d
plt.hist(genuine, bins=50, alpha=0.6, label='Genuine_Scores', color='blue', de
plt.axvline(lowest_genuine, color='blue', linestyle='--', label=f'Lowest_Genui
plt.axvline(highest_impostor, color='red', linestyle='--', label=f'Highest_Imp
plt.title("Score_Distributions:_Genuine_vs_Impostor")
plt.xlabel("Similarity_Score")
plt.ylabel("Density")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

